i18n

Práctica 9: Internacionalización (i18n) con JavaScript

Tabla de contenido

1. INTRODUCCIÓN **2**

2. OBJETIVOS DE LA PRÁCTICA **2**

3. EJERCICIOS PROPUESTOS **3**

3.1. PUNTO DE PARTIDA: VINCULAR EL SCRIPT E INICIALIZAR EL ESQUELETO DEFENSIVO 3
3.2. IDENTIFICAR LOS TEXTOS TRADUCIBLES DEL SITIO 4
Añadir atributos data-i18n al HTML *5*
Organizar los textos por claves semánticas *6*
3.3. CREAR EL DICCIONARIO DE TRADUCCIONES EN JAVASCRIPT 7
Estructura por idioma *8*
Buenas prácticas para claves y mensajes *8*
3.4. APLICAR LAS TRADUCCIONES AL DOM 8
Leer claves desde el HTML *8*
Actualizar el contenido dinámicamente *9*
Probando la traducción *10*
3.5. SELECTOR DE IDIOMA Y PERSISTENCIA DE LA PREFERENCIA 10
Añadir un selector visible en la interfaz *11*
Guardar el idioma en localStorage *11*
Restaurar el idioma al recargar o cambiar de página *12*
Conectar el selector *12*
Inicializar todo al cargar la página *13*
3.6. FORMATEO REGIONAL CON INTL 14
Fechas y horas *14*
Números y precios *14*
3.7. INTERNACIONALIZAR CONTENIDO DINÁMICO DEL CLUB DE LECTURA 16
3.8. MEJORAS DE ACCESIBILIDAD Y USABILIDAD 18
Actualizar el atributo lang del documento *18*
Evitar inconsistencias entre idiomas *18*
Ajustes visuales *18*

4. DEPURA Y CORRIGE TU CÓDIGO JS **19**

En VS Code *19*
En el navegador *20*

5. QUÉ HEMOS APRENDIDO EN ESTA PRÁCTICA **21**

1. Introducción

En esta práctica de laboratorio vas a ampliar el sitio web multipágina que has venido desarrollando en las tareas semanales, para añadir internacionalización del lado del cliente mediante JavaScript.

El objetivo es que el sitio pueda mostrarse en varios idiomas sin duplicar páginas HTML. Para ello, separarás los textos traducibles del marcado, crearás un pequeño diccionario en JavaScript, aplicarás las traducciones al DOM, guardarás la preferencia del usuario y utilizarás la API Intl para adaptar fechas, números y precios a la configuración regional activa.

Al finalizar la práctica, tu sitio no solo tendrá contenido estático en más de un idioma, sino que también mostrará información dinámica adaptada a la *locale* seleccionada, como la fecha del próximo encuentro del club de lectura o el precio de una actividad.

2. Objetivos de la práctica

Al finalizar esta sesión de prácticas, serás capaz de:

- Vincular archivos JavaScript en un proyecto web multipágina de VS Code.
- Usar un esqueleto defensivo para que el script no produzca errores en páginas que todavía no estén preparadas para la internacionalización.
- Identificar y marcar textos traducibles en el HTML mediante atributos data-i18n.
- Crear y mantener un diccionario simple de traducciones en JavaScript.
- Aplicar traducciones al DOM en función del idioma activo.
- Añadir un selector de idioma a la interfaz.
- Guardar y restaurar la preferencia de idioma con localStorage.
- Usar la API Intl para formatear fechas, horas, números y precios según la locale.
- Internacionalizar contenido dinámico del proyecto Club de Lectura.
- Verificar que la interfaz sigue siendo usable, coherente y accesible tras el cambio de idioma.

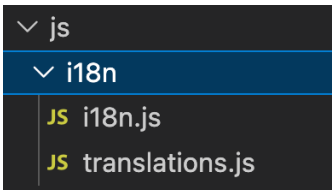
3. Ejercicios propuestos

Comienza abriendo la carpeta de tu proyecto de la Tarea 8 en VS Code. El proyecto debe contener al menos tres páginas HTML, con index.html ubicado en la carpeta raíz. Asegúrate de tener una página con formulario (por ejemplo contact.html) con entradas: texto, email, textarea, select, casillas de verificación y botones de opción, y atributos required cuando corresponda. Mantén tus archivos CSS (css/styles.css y el opcional css/single-page.css) sin cambios salvo donde sea necesario para estilos de retroalimentación (.valid, .invalid, .btn, etc.). Mantén tus archivos JS (js/MyJs.js) implementados durante la tarea 8.

Si no completaste la **Tarea 8**, puedes usar en su lugar el **proyecto plantilla de VS Code ("Club de Lectura")** proporcionado para esta sesión.

3.1. Punto de partida: vincular el script e inicializar el esqueleto defensivo

Dentro de la carpeta `js`, crea un nuevo directorio llamado i18n y dentro de él crea dos archivos, llamados `i18n.js` y `translations.js`.



Añade esta etiqueta `script` a las páginas HTML que vayas a internacionalizar:

```
    <script defer
src="js/i18n/translations.js"></script>
    <script defer src="js/i18n/i18n.js"></script>
```

Si la página está en una **subcarpeta**, ajusta la ruta:

```
  <script defer
src="../js/i18n/translations.js"></script>
  <script defer src="../js/i18n/i18n.js"></script>
```

Cargamos los scripts con `defer`, de este modo, el navegador descargará los archivos sin bloquear el parseo del HTML y los ejecutará cuando el documento ya haya sido procesado.

Igual que en la práctica anterior, es importante comenzar con un **esqueleto defensivo.** El archivo `i18n.js` puede cargarse en páginas donde todavía no existan elementos marcados para traducción o donde el selector de idioma aún no se haya añadido, por lo que el script debe comprobar esta situación antes de continuar. Si no encuentra ninguno de esos

elementos, lo más recomendable es detener la inicialización de forma limpia para evitar comprobaciones o acciones innecesarias.

Ejemplo:

```
'use strict';

function initializeI18n() {
      const translatableElements = document.querySelectorAll('[data-
i18n]');
      const languageSwitcher = document.querySelector('#language-
switcher');

      if (translatableElements.length === 0 && !languageSwitcher) {
            console.log('Esta página todavía no está preparada para
i18n.');
            return;
      }
}

initializeI18n();
```

Este tipo de comprobación evita errores y hace que el código sea más robusto.

Ejercicio 1: Añade i18n.js al proyecto e inclúyelo en las páginas del Club de Lectura. En el script, comprueba si existen elementos traducibles o un selector de idioma antes de continuar.

3.2. Identificar los textos traducibles del sitio

La internacionalización de un sitio web no consiste simplemente en traducir palabras sueltas, sino en preparar la interfaz para que los textos visibles y los mensajes principales puedan adaptarse con facilidad a distintos idiomas. Antes de escribir el código JavaScript o construir el diccionario de traducciones, conviene identificar con claridad qué fragmentos del sitio deberán cambiar cuando el usuario seleccione otra lengua. En esta práctica, ese análisis previo es importante porque después **esos textos se marcarán en el HTML con atributos data-i18n,** que podrán leerse desde JavaScript mediante la propiedad `dataset`. En un sitio web como el Club de Lectura, normalmente serán traducibles:

- El título principal
- Los enlaces de navegación
- Los subtítulos

- Los botones
- Los textos breves de presentación
- Los encabezados de secciones
- Etiquetas de formularios sencillos
- Mensajes breves del sitio

En cambio, no deben considerarse textos traducibles los nombres de clases, los identificadores, las rutas de los enlaces o cualquier dato técnico que no forme parte del contenido mostrado al usuario.

Una forma sencilla y mantenible de preparar estos textos para su traducción consiste en añadir a cada elemento un atributo **data-i18n**. Este atributo actúa como una marca que permite a JavaScript localizar el texto y sustituirlo por su equivalente en el idioma activo. En lugar de depender del contenido literal escrito en el HTML, cada fragmento quedará asociado a una clave semántica, lo que facilitará mucho el mantenimiento del sitio a medida que el proyecto crezca.

Añadir atributos data-i18n al HTML

El primer paso práctico consiste en revisar las páginas del sitio y añadir estos atributos a los elementos que contengan texto traducible. Observa el siguiente ejemplo:

```
<header>
        <h1 data-i18n="site.title">Club de Lectura</h1>
        <nav>
            <ul>
                <li><a href="index.html" data-
i18n="navigation.home">Inicio</a></li>
                <li><a href="index.html#books" data-
i18n="navigation.books">Libros</a></li>
                <li><a href="pages/eventos.html" data-
i18n="navigation.events">Encuentros</a></li>
                <li><a href="pages/contacto.html" data-
i18n="navigation.contact">Contacto</a></li>
            </ul>
        </nav>
    </header>

<section>
    <h2 data-i18n="home.welcomeTitle">Bienvenido al Club de
Lectura</h2>
    <p data-i18n="home.welcomeText">
        Descubre nuevas historias, comparte impresiones y participa en
nuestros encuentros.
    </p>
</section>
```

En este fragmento, cada texto visible queda identificado mediante una clave. Gracias a ello, más adelante será posible recorrer el documento con JavaScript, leer esas claves y reemplazar automáticamente su contenido por la traducción correspondiente. Observa además que el HTML sigue siendo legible: el texto original continúa presente, pero ahora queda preparado para ser gestionado de forma más flexible.

Principios de nomenclatura aplicados:

- Claves basadas en significado, no en texto
  - Evitar: `aboutus`, `bookswelove`.
- Jerarquía por dominio funcional
  - Algunas claves describen partes globales del sitio, como `site.*` o `navigation.*`; otras se organizan por sección o bloque funcional, como `home.*` o `events.monthlyMeeting.*`.
- Consistencia gramatical
  - Usar `camelCase` o `kebab-case`, pero no mezclarlos.
- Separar el contenido estructural de los elementos HTML
  - Evitar usar `heading`, `paragraph`, etc.

Organizar los textos por claves semánticas

Al definir estas claves, es importante elegir nombres claros y con significado. No conviene utilizar etiquetas ambiguas como **`text1`**, **`title2`** o **`paragraphA`**, porque ese tipo de nombres dificulta el mantenimiento del proyecto y vuelve más confusa la relación entre el HTML y el diccionario de traducciones. En cambio, resulta mucho más recomendable emplear claves que indiquen con precisión la zona del sitio a la que pertenece cada texto.

Ejemplo de buenas prácticas en la nomenclatura de etiquetas:

- `site.*`
- `navigation.*`
- `controls.language.*`
- `events.monthlyMeeting.*`
- `footer.*`

Antes de pasar al siguiente apartado, revisa las páginas del Club de Lectura y asegúrate de que los principales textos visibles han quedado correctamente marcados con atributos `data-i18n` y con claves bien elegidas.

Ejercicio 2: Recorre varias páginas del Club de Lectura y marca los principales textos visibles con atributos data-i18n y claves semánticas.

3.3. Crear el diccionario de traducciones en JavaScript

Una vez que los textos del HTML han quedado identificados mediante claves semánticas, el siguiente paso consiste en definir un diccionario de traducciones. Su función será asociar cada clave con el texto que debe mostrarse en cada idioma. Para esta práctica, una solución sencilla y suficiente es utilizar un objeto JavaScript organizado por idiomas.

Crea el archivo **`js/i18n/translations.js`**. Utilizamos **`window.translations`** para que el diccionario esté disponible globalmente. En el navegador, **`window`** representa el objeto global de la página.

Ejemplo:

```
  window.translations = {
    es: {
        'site.title': 'Club de Lectura',

        'navigation.home': 'Inicio',
        'navigation.books': 'Libros',
        'navigation.events': 'Encuentros',
        'navigation.contact': 'Contacto',

        'controls.language.label': 'Idioma',
        'controls.language.spanish': 'Español',
        'controls.language.english': 'Inglés',

        'events.monthlyMeeting.dateLabel': 'Fecha:',
        'events.monthlyMeeting.feeLabel': 'Cuota:',
        'events.monthlyMeeting.summaryTitle': 'Resumen del
encuentro',
    },
    en: {
        'site.title': 'Book Club',

        'navigation.home': 'Home',
        'navigation.books': 'Books',
        'navigation.events': 'Meetings',
        'navigation.contact': 'Contact',

        'controls.language.label': 'Language',
        'controls.language.spanish': 'Spanish',
        'controls.language.english': 'English',

        'events.monthlyMeeting.dateLabel': 'Date:',
        'events.monthlyMeeting.feeLabel': 'Fee:',
```

```
        'events.monthlyMeeting.summaryTitle': 'Meeting summary',
    }
};
```

Estructura por idioma

Observa que el primer nivel del objeto distingue los idiomas disponibles, en este caso es y en. Dentro de cada uno de ellos aparecen pares clave-valor, donde la clave identifica un texto de la interfaz y el valor contiene su traducción. Es importante que una misma clave exista en todos los idiomas definidos, ya que de lo contrario algunos fragmentos del sitio no podrán traducirse correctamente.

Buenas prácticas para claves y mensajes

Conviene mantener la misma estructura de claves en todos los idiomas y utilizar nombres que indiquen con claridad a qué parte del sitio pertenece cada texto. También es recomendable escribir mensajes naturales y completos, evitando claves demasiado genéricas o nombres técnicos que no reflejen el contenido real que verá el usuario. Cuanto más ordenado esté el diccionario desde el principio, más fácil será ampliarlo y corregirlo después.

Ejercicio 3: Crea un diccionario JavaScript en (js/i18n/translations.js) con al menos dos idiomas (es y en) y suficientes claves para traducir la cabecera, la navegación y varios bloques principales del Club de Lectura.

3.4. Aplicar las traducciones al DOM

Una vez definidos el HTML y el diccionario, necesitas una función que recorra los elementos marcados con data-i18n y actualice su texto según el idioma activo.

Leer claves desde el HTML

Cada elemento que has preparado en el marcado guarda su clave en el atributo data-i18n. Desde JavaScript, esa información puede recuperarse mediante dataset.i18n, lo que permite saber qué entrada del diccionario debe utilizarse en cada caso.

Actualizar el contenido dinámicamente

Como el archivo translations.js se carga antes que i18n.js, el diccionario de traducciones ya estará disponible cuando se ejecute el script principal.

Ejemplo:

```
'use strict';

function applyTranslations(locale) {
    const translatableElements =
document.querySelectorAll('[data-i18n]');
    const labelElements = document.querySelectorAll('[data-i18n-
label]');
    const dict = window.translations?.[locale];

    for (let i = 0; i < translatableElements.length; i++) {
        const element = translatableElements[i];
        const key = element.dataset.i18n;
        const translatedText = dict[key];

        if (translatedText) {
            element.textContent = translatedText;
        }
    }

   for (let i = 0; i < labelElements.length; i++) {
        const element = labelElements[i];
        const key = element.dataset.i18nLabel;
        const translatedLabel = dict[key];

        if (translatedLabel) {
            element.setAttribute('label', translatedLabel);
        }
    }
    document.documentElement.lang = locale;
}

// Si también se quisieran traducir placeholders, habría que
// marcarlos con otro atributo, por ejemplo, data-i18n-
placeholder,
// y recorrerlos en un segundo bucle para actualizar su atributo
placeholder.
// element.setAttribute('placeholder', translatedPlaceholder);
```

Nota: En este código se utiliza el operador **optional chaining (operador ?.),** si **translations[locale]** es **null** o **undefined**, devuelve **undefined** sin lanzar error. Si existe, continúa accediendo a su clave.

Qué hace esta función:

La función busca todos los elementos marcados con `data-i18n`, lee la clave de cada uno, consulta esa clave en el diccionario del idioma activo y, si encuentra una traducción, **sustituye su contenido mediante `textContent`**. Si el **texto está en algún atributo HTML** (`label`, `placeholder`, `title`, etc.) se debe hacer mediante **`setAttribute`** (segundo bucle `for` de la función). Al final, actualiza `document.documentElement.lang` con el idioma aplicado.

Probando la traducción

Para comprobar que el mecanismo funciona, puedes invocar la función indicando el idioma deseado:

```
applyTranslations('es');
```

o bien:

```
applyTranslations('en');
```

Si todo está correctamente preparado, al cambiar el valor del parámetro deberían actualizarse automáticamente los textos marcados de la interfaz. En este punto ya tendrías resuelto el mecanismo básico de traducción del sitio y podrías pasar al selector de idioma y a la persistencia de la preferencia del usuario.

Ejercicio 4: Implementa una función `applyTranslations(locale)` que lea las claves desde el HTML y actualice los textos visibles del Club de Lectura según el idioma seleccionado.

3.5. Selector de idioma y persistencia de la preferencia

Traducir el sitio manualmente desde el código no es suficiente si el usuario no puede elegir el idioma desde la propia interfaz. Por ello, en este apartado vas a incorporar un selector visible que permita cambiar de lengua de forma cómoda y, además, vas a guardar esa preferencia para que se mantenga al recargar la página o al navegar entre distintas secciones del sitio.

Añadir un selector visible en la interfaz

Una forma sencilla de resolverlo consiste en añadir un elemento select en la cabecera del sitio. De este modo, el usuario podrá elegir directamente entre los idiomas disponibles sin necesidad de modificar el código.

Ejemplo:

```
<div class="language-box">
    <label for="language-switcher" data-
i18n="common.language.label">Language</label>
    <select id="language-switcher">
        <option value="es" data-
i18n="common.language.spanish">Español</option>
        <option value="en" data-
i18n="common.language.english">English</option>
    </select>
</div>
```

Con este fragmento, la interfaz ya dispone del control necesario para cambiar de idioma. A partir de aquí, solo falta enlazar ese selector con la lógica JavaScript para aplicar el cambio y conservar la preferencia elegida.

Guardar el idioma en localStorage

Cuando el usuario seleccione un idioma, conviene almacenar esa elección en **localStorage**. Así, el navegador podrá recordar la preferencia incluso después de cerrar la página o volver a entrar más tarde.

localStorage es un mecanismo de almacenamiento del navegador que permite guardar datos en forma de pares clave-valor asociados al mismo sitio web. A diferencia de una variable normal de JavaScript, su contenido no se pierde al recargar la página, y a diferencia de sessionStorage, tampoco se elimina al cerrar la pestaña o el navegador.

```
function setLocale(locale) {
    const selectedLocale = window.translations[locale] ? locale : 'es';

    localStorage.setItem('preferredLanguage', selectedLocale);
    applyTranslations(selectedLocale);
    //updateDynamicContent(selectedLocale);
   // initializeLanguageSwitcher(selectedLocale);
}
```

```
// Nos aseguramos de que setLocale sea global.
window.setLocale = setLocale;
```

Esta función cumple dos tareas importantes: por un lado, guarda el idioma seleccionado y, por otro, actualiza tanto los textos fijos como el contenido dinámico del sitio. De este modo, el cambio resulta inmediato y coherente en toda la interfaz.

Restaurar el idioma al recargar o cambiar de página

Una vez guardada la preferencia, es necesario recuperarla automáticamente al cargar cada página. Si no existiera ninguna preferencia previa, puedes establecer el español como valor por defecto (expresión || **'es'**).

Para ello, resulta útil definir una pequeña función de apoyo:

```
function getSavedLocale() {
    const savedLocale = localStorage.getItem('preferredLanguage');
    return translations[savedLocale] ? savedLocale : 'es';
}
```

Esta función recupera el valor almacenado con la clave **`preferredLanguage`**. Si el valor existe y coincide con uno de los idiomas definidos en el diccionario, se devuelve ese idioma; en caso contrario, se utiliza “es” como alternativa segura.

Conectar el selector

Una vez recuperado el idioma guardado, el siguiente paso consiste en enlazar el selector con la función que realiza el cambio. En esta práctica, el cambio de idioma se gestionará directamente desde el HTML mediante el **atributo `onchange` del elemento `select`.**

```
    <select id="language-switcher"
onchange="setLocale(this.value)">
```

De este modo, cada vez que se seleccione un valor distinto, se ejecutará la función **`setLocale`** con el idioma elegido. Para que el selector muestre desde el principio la preferencia recuperada, basta con asignar ese valor al cargar la página:

```
function initializeLanguageSwitcher(locale) {
    const languageSwitcher = document.querySelector('#language-
switcher');

    if (languageSwitcher) {
        languageSwitcher.value = locale;
    }
}
```

Con esta modificación, el idioma podrá cambiarse directamente desde la interfaz y la selección se conservará entre páginas o recargas del sitio.

Inicializar todo al cargar la página

Para que el comportamiento quede completo, resulta conveniente reunir la inicialización en una función. Esa función comprobará si la página está preparada para i18n, recuperará el idioma guardado y aplicará la configuración correspondiente desde el principio. **Actualiza la función que teníamos inicialmente** para el esqueleto defensivo:

```
function initializeI18n() {
    const translatableElements = document.querySelectorAll('[data-
i18n]');
    const languageSwitcher = document.querySelector('#language-
switcher');

    if (translatableElements.length === 0 && !languageSwitcher) {
        console.log('Esta página todavía no está preparada para
i18n.');
        return;
    }

    const initialLocale = getSavedLocale();
    applyTranslations(initialLocale);
    //updateDynamicContent(initialLocale);
    initializeLanguageSwitcher(initialLocale);
}
```

De este modo, al cargarse la página se recupera automáticamente la preferencia guardada, se traducen los textos visibles, se actualiza el contenido dinámico y el selector muestra el idioma correcto desde el primer momento.

Ejercicio 5: Añade un selector de idioma a la cabecera del Club de Lectura, guarda la preferencia con localStorage y recupérala automáticamente al recargar o cambiar de página.

3.6. Formateo regional con Intl

La internacionalización no consiste solo en traducir palabras. También implica mostrar fechas, horas, números y moneda de forma adecuada para cada `locale` activo. Para ello, JavaScript proporciona la API **Intl**, que permite realizar este formateo de manera nativa y bastante sencilla.

Fechas y horas

Supongamos que el Club de Lectura muestra la fecha del próximo encuentro. En ese caso, no basta con traducir la etiqueta del campo, sino que también conviene adaptar la forma en la que se representa la fecha y la hora.

HTML:

```
<p>
  <span data-i18n="events.monthlyMeeting.dateLabel">Fecha:</span>
    <em id="monthly-meeting-date" data-date="2026-07-
15T19:00:00"></em>
</p>
```

JavaScript:

```
function formatDate(dateValue, locale) {
    const resolvedLocale = locale === 'en' ? 'en-US' : 'es-ES';

    return new Intl.DateTimeFormat(resolvedLocale, {
        dateStyle: 'long',
        timeStyle: 'short'
    }).format(dateValue);
}
```

Esta función recibe la fecha y la locale activa y devuelve una fecha formateada según las convenciones de ese idioma. Así, el mismo valor puede mostrarse con un aspecto distinto en español y en inglés sin necesidad de construir manualmente cada formato.

Números y precios

La misma idea puede aplicarse a cantidades económicas. Si el sitio muestra una cuota mensual o el precio de una actividad, lo adecuado es que la moneda y el formato numérico también se adapten al contexto regional.

HTML:

```
<p>
  <span data-i18n="events.monthlyMeeting.feeLabel">Cuota:</span>
        <em id="monthly-meeting-fee" data-fee="12.5"></em>
</p>
```

JavaScript:

```
function formatCurrency(amount, locale) {
    const resolvedLocale = locale === 'en' ? 'en-US' : 'es-ES';
    const currency = locale === 'en' ? 'USD' : 'EUR';

    return new Intl.NumberFormat(resolvedLocale, {
        style: 'currency',
        currency: currency
    }).format(amount);
}
```

Mostrar ambos valores:

```
function updateDynamicContent(locale) {
    const meetingDateBox = document.querySelector('#monthly-
meeting-date');
    const meetingFeeBox = document.querySelector('#monthly-meeting-
fee');

    if (meetingDateBox) {
        const rawDate = meetingDateBox.dataset.date;
        const meetingDate = new Date(rawDate);
        meetingDateBox.textContent = formatDate(meetingDate,
locale);
    }

    if (meetingFeeBox) {
        const rawFee = Number(meetingFeeBox.dataset.fee);
        meetingFeeBox.textContent = formatCurrency(rawFee, locale);
    }
}
```

En este caso, la función decide qué moneda utilizar y devuelve el importe formateado correctamente. El resultado no depende solo del texto visible, sino también de las reglas de representación numérica propias de cada locale.

Ejercicio 6: Usa Intl.DateTimeFormat e Intl.NumberFormat para mostrar en el Club de Lectura, al menos, una fecha y un precio o cuota adaptados al idioma activo.

3.7. Internacionalizar contenido dinámico del Club de Lectura

Hasta ahora has trabajado con textos estáticos y con datos regionales formateados. El siguiente paso consiste en aplicar esa misma lógica a contenido generado dinámicamente, es decir, a fragmentos que no están escritos de forma fija en el HTML, sino que se construyen desde JavaScript.

Una buena opción es crear un pequeño bloque informativo del club, por ejemplo, en eventos.html o en la página principal.

Podemos añadir una nueva `section` después de la primera sección de eventos.html:

```
<section class="club-info">
    <h2 data-i18n="events.monthlyMeeting.summaryTitle">
                Próximo encuentro </h2>
    <p id="meeting-summary"></p>
</section>
```

A partir de aquí, puede definirse una función que construya un texto distinto según el idioma activo. Para ello, se leerán desde el HTML la fecha y la cuota ya formateadas, así como otros textos del propio documento, como el lugar del encuentro y el título del libro, y después se generará una frase completa en español o en inglés.

```
function buildMeetingSummary(locale) {
    const meetingDateBox = document.querySelector('#monthly-
meeting-date');
    const meetingFeeBox = document.querySelector('#monthly-meeting-
fee');
    const meetingPlaceBox = document.querySelector('[data-
i18n="events.monthlyMeeting.locationValue"]');
    const bookTitleBox = document.querySelector('[data-
i18n="events.monthlyMeeting.bookTitle"]');

    const formattedDate = meetingDateBox ?
meetingDateBox.textContent : '';
    const formattedFee = meetingFeeBox ? meetingFeeBox.textContent
: '';
    const place = meetingPlaceBox ? meetingPlaceBox.textContent :
'';
    const bookTitle = bookTitleBox ? bookTitleBox.textContent : '';

    if (locale === 'en') {
        return `Our next monthly meeting will take place on
${formattedDate} at ${place}. We will discuss ${bookTitle}, and the
current fee is ${formattedFee}.`;
    }
```

```
    return `Nuestro próximo encuentro mensual será el
${formattedDate} en ${place}. Hablaremos sobre ${bookTitle} y la
cuota actual es ${formattedFee}.`;
}
```

En este caso, la función no solo cambia algunas palabras aisladas, sino que genera un bloque completo usando valores que ya han sido traducidos o formateados previamente. Las plantillas literales de JavaScript resultan útiles aquí porque permiten interpolar expresiones dentro de una cadena.

Para mostrar ese resumen en la página, basta con **ampliar** la versión anterior de la función **`updateDynamicContent(locale)`:**

```
function updateDynamicContent(locale) {
    const meetingDateBox = document.querySelector('#monthly-
meeting-date');
    const meetingFeeBox = document.querySelector('#monthly-
meeting-fee');
    const meetingSummary = document.querySelector('#meeting-
summary');

    if (meetingDateBox) {
        const rawDate = meetingDateBox.dataset.date;
        const meetingDate = new Date(rawDate);
        meetingDateBox.textContent = formatDate(meetingDate,
locale);
    }

    if (meetingFeeBox) {
        const rawFee = Number(meetingFeeBox.dataset.fee);
        meetingFeeBox.textContent = formatCurrency(rawFee,
locale);
    }

    if (meetingSummary) {
        meetingSummary.textContent = buildMeetingSummary(locale);
    }
}
```

Con esta ampliación, el Club de Lectura ya no se limita a traducir títulos, menús o etiquetas, sino que también adapta bloques completos generados con JavaScript.

Ejercicio 7: Añade al menos un bloque dinámico del Club de Lectura cuyo contenido cambie según el idioma activo y use además fechas o precios formateados con Intl.

3.8. Mejoras de accesibilidad y usabilidad

Una internacionalización correcta no debe limitarse a sustituir cadenas de texto. También debe mantener la claridad de la interfaz, la coherencia entre elementos y unas condiciones mínimas de accesibilidad. Por ello, una vez implementado el cambio de idioma, conviene revisar cómo se comporta el sitio en conjunto.

Actualizar el atributo lang del documento

Ya hemos incluido esto en applyTranslations(locale):

```
document.documentElement.lang = locale;
```

Este detalle es importante porque ayuda a que navegadores y tecnologías de asistencia interpreten correctamente el idioma principal de la página. Aunque es una modificación pequeña, forma parte de una implementación más correcta y completa.

Evitar inconsistencias entre idiomas

Al probar la aplicación, asegúrate de que el cambio afecta de forma coherente a toda la interfaz. La navegación, los títulos, los subtítulos y los mensajes principales deben actualizarse de forma uniforme, y las fechas o cantidades deben mostrarse de acuerdo con la locale seleccionada. También conviene comprobar que no quedan bloques mezclados en español e inglés, salvo en zonas que todavía no se hayan preparado.

Ajustes visuales

El cambio de idioma también puede afectar a la presentación visual. Algunas expresiones en inglés ocupan más espacio que sus equivalentes en español, mientras que otras pueden ocupar menos. Por ello, merece la pena revisar botones, menús, cabeceras, tarjetas estrechas o etiquetas relativamente largas para comprobar que el diseño sigue siendo claro y usable.

Si lo necesitas, puedes añadir un pequeño ajuste de estilo al selector de idioma para integrarlo mejor en la cabecera:

```
.language-box {
    display: inline-flex;
    align-items: center;
    gap: 0.5rem;
    margin-left: 1rem;
}

.language-box select {
    padding: 0.3rem 0.5rem;
    font-size: 1rem;
}
```

Con estas comprobaciones finales, nos aseguramos de que el cambio de idioma no rompe la experiencia de navegación y de que la interfaz sigue siendo flexible, clara y adecuada para distintos usuarios.

Ejercicio 8: Revisa que el sitio siga siendo usable tras el cambio de idioma, actualiza correctamente el atributo lang y comprueba que no aparezcan mezclas incoherentes de idiomas en la interfaz.

4. Depura y corrige tu código JS

Una vez completada la funcionalidad principal de la práctica, conviene revisar el código para detectar posibles errores de sintaxis, referencias incorrectas o comportamientos no previstos. Para ello, puedes apoyarte tanto en el editor como en las herramientas de desarrollo del navegador, igual que en la práctica anterior.

En VS Code

Comprueba si hay:

- Llaves, comillas o paréntesis que faltan
- Nombres de clave mal escritos
- Errores en rutas del script
- Variables declaradas, pero no usadas
- Selectores incorrectos

Muchos problemas en esta práctica no se deben a errores complejos, sino a pequeños desajustes entre el HTML y el código JavaScript. Por ello, merece la pena comprobar con calma que los identificadores, los atributos data-i18n y los nombres de las claves coincidan exactamente en todos los archivos.

En el navegador

Después de esta primera revisión, abre las Developer Tools del navegador y consulta la consola. Ahí podrás comprobar si el script se carga correctamente, si el selector de idioma responde al cambiar de valor y si los textos visibles se actualizan como esperas.

Comprueba si:

- El script se carga correctamente
- El selector de idioma responde
- Los textos cambian al seleccionar otro idioma
- La preferencia se conserva tras recargar
- Fechas y precios cambian con la locale
- No aparecen errores por elementos inexistentes

5. Qué hemos aprendido en esta práctica

- Cómo vincular un archivo JS externo e inicializarlo de forma segura
- Cómo marcar textos traducibles en el HTML con atributos data-i18n
- Cómo organizar un diccionario de traducciones por idioma
- Cómo aplicar traducciones al DOM dinámicamente
- Cómo permitir al usuario cambiar de idioma desde la interfaz
- Cómo guardar la preferencia con localStorage
- Cómo usar Intl para formatear fechas, horas y precios
- Cómo internacionalizar también contenido dinámico del sitio
- Cómo revisar la coherencia, la usabilidad y la accesibilidad tras el cambio de idioma